

BriskSeed: Online History-Guided Reuse for Accelerating Dynamic Approximate Nearest Neighbor Search

Hongru Gao

HUST

Wuhan, China

hrgao@hust.edu.cn

ABSTRACT

Approximate Nearest Neighbor Search (ANNS) is a core primitive in modern data systems, but most high-performance graph-based indices are optimized for static workloads rather than continuously evolving data. Under streaming insertions and deletions, pre-computed search guidance quickly becomes stale, causing either lower query efficiency or expensive re-optimization. To address this challenge, we present BriskSeed, a pluggable optimization module for dynamic ANNS. Its key idea is to turn past successful search outcomes into reusable search primitives, referred to as *seeds*, while controlling the overhead and risk of stale reuse under evolving data. Specifically, BriskSeed integrates seed storage, seed-guided search, and reliability control into a unified online workflow: it organizes seeds in a two-tier utility-aware storage scheme, activates them through exact lookup and bounded semantic probing, and validates them through a lightweight acceptance-and-fallback policy. This design preserves a compact set of high-value seeds under distribution drift while remaining compatible with existing ANNS backends. We evaluate BriskSeed on **CANDOR-Bench**, a streaming ANNS benchmark framework, across multiple ANNS backends under a unified plugin interface. Experimental results show that BriskSeed consistently improves end-to-end search efficiency while sustaining competitive recall under high update rates.

PVLDB Reference Format:

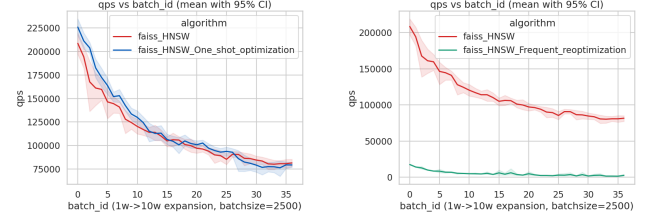
Hongru Gao. BriskSeed: Online History-Guided Reuse for Accelerating Dynamic Approximate Nearest Neighbor Search. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Approximate Nearest Neighbor Search (ANNS) is a core primitive in modern data systems, including Retrieval-Augmented



(a) One-shot optimization (b) Frequent re-optimization

Figure 1: Trade-offs in dynamic ANNS: one-shot optimization provides only short-term gains, while aggressive frequent re-optimization hurts overall query throughput.

Generation (RAG), recommendation, and anomaly detection [8]. Among existing ANNS index families, graph-based indices such as HNSW [4, 5] are widely adopted because they offer strong recall-latency trade-offs at scale. Their search process relies on graph traversal over non-contiguous memory regions, where random memory access and cache misses often dominate runtime. To improve query performance, prior work generally follows two directions: system-level optimizations that make distance computation and traversal faster through optimized storage layout, SIMD, quantization techniques, etc [3, 7, 8], and algorithm-level optimizations that reduce the amount of computation through pruning and distance estimation [1, 2, 6].

Despite substantial progress, most existing optimizations are developed for static snapshots. In practical streaming systems, vectors are continuously inserted and deleted, causing the index topology and data distribution to evolve over time. This creates a temporal mismatch: pre-computed search hints, i.e., auxiliary information or structures used to accelerate queries, may be effective at time t but become stale at time $t + \Delta$, reducing alignment with the current index and degrading query efficiency. The effect is especially severe in high-rate ingestion workloads, where updates arrive continuously and drift accumulates quickly. A representative case is real-time payment fraud detection, where incoming transactions are matched against a behavioral embedding database via ANNS under millisecond-level decision SLAs. In such systems, significant performance degradation across updates is unacceptable because delayed retrieval directly increases fraud leakage risk and financial loss.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

To alleviate update-induced performance decay, dynamic ANNS faces a fundamental efficiency-maintenance dilemma, as illustrated in Figure 1. If search hints are computed once and left unmaintained during updates, they may provide short-term gains in early rounds, but their benefit decays over time, causing query throughput to progressively fall back toward baseline performance (Figure 1a). In contrast, if the system frequently rebuilds or globally re-optimizes them, the maintenance overhead is prohibitive and can dominate end-to-end runtime, limiting overall throughput gains (Figure 1b). Consequently, the key challenge is not merely how to accelerate search, but how to keep acceleration *continuously valid* under ongoing updates.

To address this challenge, we propose StreamSeed, a dynamic acceleration framework for streaming ANNS. The key idea is to turn one-shot offline acceleration into an online control loop that continuously reuses, validates, and refreshes historical search outcomes as the workload and index evolve. Our design is motivated by the observation that, for the same query batch, the top- k results before and after moderate updates often still overlap substantially and the searched vector distributions exhibit a distinct long-tail nature. Based on these insights, StreamSeed maintains a bounded seed store as reusable incremental state and continuously writes back effective seeds from recent queries. At query time, it retrieves seeds through exact lookup or bounded semantic probing, performs bounded seed-guided expansion, and applies an acceptance-and-fallback gate to prevent unreliable reuse from degrading recall. To keep the design lightweight and practical, StreamSeed further decouples its control policy from backend-specific index internals, which enables integration with different dynamic ANN backends at low engineering cost.

This paper makes the following contributions:

- We identify stale-guidance degradation in streaming ANNS and formulate it as an amortized search-maintenance optimization problem, which explains why static accelerations lose effectiveness under continuous updates.
- We present StreamSeed, a dynamic acceleration framework that incrementally reuses and refreshes seeds under workload drift while remaining decoupled from backend-specific index internals.
- We implement StreamSeed on CANDOR-Bench and evaluate it through extensive experiments and ablations. The results show that StreamSeed improves end-to-end efficiency while maintaining competitive recall under high update rates.

The rest of this paper is organized as follows. Section 2 formalizes the problem and preliminaries. Section 3 presents the StreamSeed design. Section 4 describes implementation details. Section 5 reports evaluation and ablation results. Section 6 discusses related work, and Section 7 concludes this work.

2 PROBLEM FORMULATION AND MOTIVATION

This section first formulates the problem statement in Section 2.1. Section 2.2 then presents two key observations that motivate our design, namely temporal locality across updates and skew in practical query workloads. Finally, Section 2.3 discusses the main challenges in designing BriskSeed.

2.1 Problem Statement

We begin with the standard Approximate Nearest Neighbor Search (ANNS) definition and then extend it to the dynamic setting considered in this paper.

Table 1: Summary of terminologies

Symbol	Description
$\mathcal{D}$	Vector dataset.
$\mathcal{D}_r$	Vector dataset state after applying the batch updates at round r .
$\hat{N}_k(q)$	Approximate top- k result set of query q .
$N_k^*(q)$	Exact top- k nearest-neighbor set of query q .
$\text{Recall}@k(q)$	Recall of the approximate top- k result for query q .
$\mathcal{U}_r$	Batch updates at round r .
$\mathcal{Q}_r$	Batch queries executed at round r .
$\mathcal{R}$	The number of rounds we evaluate.
$L_{\text{search}}(q)$	Search latency of query q .
$L_{\text{maintain}}(\mathcal{U}_r)$	Maintenance cost triggered by batch updates $\mathcal{U}_r$.
$\text{QPS}(R)$	Average query throughput within R rounds.
τ	Tolerance threshold for $\text{Recall}@k$.

Definition 1 (ANNS). Given a vector dataset $\mathcal{D} \subset \mathbb{R}^d$, a query $q \in \mathbb{R}^d$, and an integer k , ANNS returns an approximate top- k set $\hat{N}_k(q)$ that approximates the exact nearest-neighbor set $N_k^*(q)$ under a distance metric. The standard search quality metric is $\text{Recall}@k$:

$$\text{Recall}@k(q) = \frac{|\hat{N}_k(q) \cap N_k^*(q)|}{k}. \quad (1)$$

Definition 1 captures the static ANNS setting, where the vector dataset and index structure remain fixed during queries. In practical systems, however, vectors are continuously inserted and deleted, and therefore the ANNS index also evolves over time. To model this setting, we extend ANNS to a dynamic formulation and adopt the serialized round-based execution model used in CANDOR-Bench.

Definition 2 (Dynamic ANNS). Dynamic ANNS operates on a time-evolving dataset in rounds. At each round r , the system first applies a batch updates $\mathcal{U}_r$ (insertions/deletions) to transform $\mathcal{D}_{r-1}$ into $\mathcal{D}_r$, and then executes a batch queries $\mathcal{Q}_r$ on $\mathcal{D}_r$. The index structure is updated online across rounds.

In this setting, many optimizations developed for static ANNS become less effective as updates accumulate, including

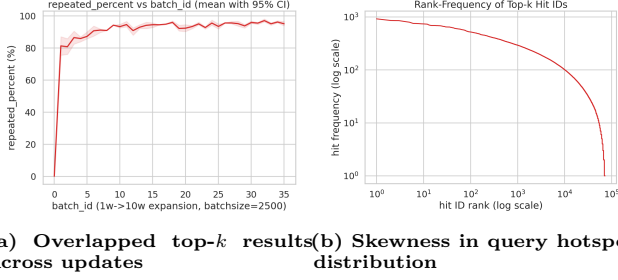


Figure 2: Key observations in dynamic ANNS.

layout restructuring, quantization-based acceleration, and globally precomputed auxiliary guidance. As their effectiveness decays, query throughput drops and recall may also deteriorate. Frequent re-optimization is not a viable solution either, because its maintenance overhead can dominate end-to-end cost, as shown in Figure 1b. This dilemma motivates the following problem.

Problem Statement. The goal is to maximize end-to-end query throughput in dynamic ANNS under continuous updates, while preserving comparable recall to baselines. In dynamic ANNS, low query latency is meaningful only if the maintenance required to sustain it is also affordable. We therefore account for both search cost and maintenance cost in the end-to-end throughput objective across update-query rounds.

Let $\mathcal{Q}_r$ denote the query batch at round r , and let $n_r = |\mathcal{Q}_r|$ be the number of queries in that round. Over an evaluation horizon of R rounds, we define the total search cost and total maintenance cost as

$$L_s(R) = \sum_{r=1}^R \sum_{q \in \mathcal{Q}_r} L_{\text{search}}(q), \quad L_m(R) = \sum_{r=1}^R L_{\text{maintain}}(\mathcal{U}_r). \quad (2)$$

The corresponding query throughput is

$$\text{QPS}(R) = \frac{\sum_{r=1}^R n_r}{L_s(R) + L_m(R)}. \quad (3)$$

Our objective is to maximize $\text{QPS}(R)$ subject to an average recall constraint:

$$\begin{aligned} \max \quad & \text{QPS}(R) \\ \text{s.t.} \quad & \frac{1}{\sum_{r=1}^R n_r} \sum_{r=1}^R \sum_{q \in \mathcal{Q}_r} \text{Recall}@k(q) \geq \tau \cdot \overline{\text{Recall}}^{\text{baseline}}, \end{aligned} \quad (4)$$

where

$$\overline{\text{Recall}}^{\text{baseline}} = \frac{1}{\sum_{r=1}^R n_r} \sum_{r=1}^R \sum_{q \in \mathcal{Q}_r} \text{Recall}@k(q)^{\text{baseline}}, \quad (5)$$

and $\tau \in (0, 1]$ controls the required recall ratio relative to the baseline.

2.2 Key Observations

To better understand the behavior of dynamic ANNS workloads, we conduct a preliminary empirical study and identify two key observations.

Observation 1: Temporal locality across updates. For the same query batch, the top- k results often remain highly overlapping across nearby update rounds. As shown in Figure 2a, the overlap ratio of top- k results quickly exceeds 80% after the first few batches and remains around 95% thereafter. This pattern indicates a form of *temporal locality across updates*: although accumulated updates may gradually reduce the effectiveness of search acceleration, they do not immediately cause large changes in the query results. Therefore, recent high-quality search outcomes can potentially be reused for subsequent queries, as long as their reuse is bounded and adaptive.

Observation 2: Long-tail skew in query-hit frequency. Across batch queries, the top- k hit IDs often exhibit strong concentration rather than uniform coverage. As shown in Figure 2b, the frequency of hit IDs follows a clear long-tail distribution: a small fraction of IDs is found very frequently, while the majority is rarely observed. This skew indicates that query-hit patterns in dynamic ANNS are highly uneven, and that not all recurring search outcomes are equally valuable for reuse. As a result, treating historical outcomes discriminately is likely to be cost-effective.

Taken together, these observations reveal both an opportunity and a principle for dynamic acceleration. Temporal locality suggests that recent search outcomes can be reused across updates, while long-tail skew suggests that such reuse should be selective rather than uniform. Therefore, an effective dynamic ANNS system should support both online reuse and cost-aware prioritization.

2.3 Design Challenges

Although the observations above reveal clear opportunities for dynamic ANNS acceleration, exploiting them efficiently and robustly under continuous updates remains non-trivial, with the following challenges:

Challenge 1: How should historical search outcomes be represented and stored? Although Observation 1 suggests that recent top- k results can remain useful across nearby update rounds, high overlap does not imply that past outcomes can be reused unchanged. Instead, they should be retained as lightweight guidance for future search. Therefore, the system must decide what information should be preserved from previous queries, how to represent it compactly, and how to store it under bounded memory overhead. Moreover, it must also balance retention and eviction so that useful outcomes remain available while stale ones are removed over time.

Challenge 2: How can historical outcomes guide the search process? Even when historical outcomes are stored, turning them into effective search guidance remains non-trivial. The system must activate relevant outcomes for queries that do not exactly match previous ones, and use

them to recover strong candidates without causing excessive search expansion. Moreover, Observation 2 suggests that reuse should be selective rather than uniform: highly recurring outcomes and long-tail outcomes may need to be handled differently in order to achieve good performance under bounded cost. But, how to design such a selective reuse mechanism that can adapt to changing query-hit patterns is an open question.

Challenge 3: How can historical-outcome reuse improve end-to-end query efficiency while preserving recall? Even when historical outcomes are well represented and selectively prioritized for search, reuse does not automatically improve both overall performance and recall, because the identified historical outcomes may still not reliable for current search. The system must therefore decide whether the reused search path should be accepted or whether it should fall back to the native search path, while keeping this decision process sufficiently lightweight. Designing such an acceptance-and-fallback mechanism is critical to ensuring that historical-outcome reuse improves efficiency without degrading recall.

Motivated by these challenges, we next present our design BriskSeed and elaborate on how it address Challenges 1-3.

3 BRISKSEED DESIGN

This section presents the full BriskSeed design for dynamic ANNS under continuous updates. Section 3.1 introduces the design overview. Section 3.2 describes the overall pipeline of BriskSeed. Section 3.3 gives complexity, robustness, and challenge-wise implications.

3.1 Design Overview

Motivated by the observations and challenges in Section 2, we design **BriskSeed** as a three-layer online framework for accelerating dynamic ANNS. BriskSeed is built on a simple principle: historical search outcomes, referred to as *seeds*, should be reused only when they can be represented compactly, activated selectively, and trusted safely under continuous updates. To this end, BriskSeed combines three layers that map directly to the three challenges in Section 2: Seed Storage Layer, Seed-Guided Search Layer, and Reliability Control Layer, as shown in Figure 3.

(1) Seed Storage Layer (addresses Challenge 1). This layer determines how historical outcomes are represented, retained, and refreshed under bounded memory. To exploit the high top- k overlap across nearby rounds, BriskSeed organizes reusable history in a two-tier seed storage: a hot-exact tier for highly recurring outcomes and a semantic-shared tier for compact long-tail coverage. Each seed stores historical top- k IDs together with lightweight metadata needed for retrieval and maintenance. In this way, past outcomes are encoded as reusable guidance rather than final answers, while retention and eviction remain explicitly budgeted.

(2) Seed-Guided Search Layer (addresses Challenge 2). This layer determines how stored outcomes guide query-time search. For each query, BriskSeed first retrieves seeds

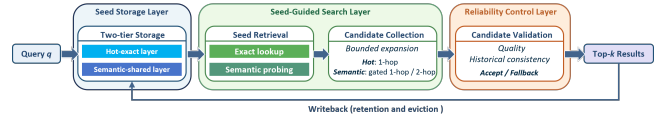


Figure 3: BriskSeed design overview.

through exact lookup or bounded semantic probing, and then converts the selected seed into provisional candidates through bounded expansion. The guidance strategy is tier-aware: hot-exact seeds use one-hop neighbor expansion by default, while semantic-shared seeds use guarded multi-hop expansion only when one-hop evidence is insufficient. This design enables efficient reuse under skewed workloads while preventing uncontrolled search overhead.

(3) Reliability Control Layer (addresses Challenge 3). This layer ensures that reuse improves end-to-end efficiency without degrading recall. Recovered candidates are checked by a lightweight query-level acceptance policy, where acceptance requires both strong local candidate quality and consistency with the seed’s historical neighborhood. If either signal is weak, BriskSeed immediately falls back to the native search path. This explicit reliability gate ensures that acceleration remains beneficial while bounding recall risk.

These three layers are co-designed rather than independent. The seed storage layer determines what is reusable, the seed-guided search layer determines how reuse enters search, and the reliability control layer determines when recovered candidates are trustworthy enough to be accepted. Together, they form a bounded online reuse framework that aligns directly with the three design challenges and remains robust under continuous updates.

3.2 Unified Online Reuse Pipeline

In this subsection, we detail the modules of BriskSeed and their interactions in the unified online reuse pipeline. Important symbols are summarized in Table 2.

3.2.1 Seed representation and storage.

Seed representation. We first define the representation of a seed, and then describe how such seeds are organized in the storage.

Each seed is stored as

$$r = \langle \mathbf{z}, n_b, \beta, \rho \rangle, \quad (6)$$

where $\mathbf{z} \in \mathbb{N}^k$ is the historical top- k result IDs used as reuse anchors, n_b is the seed birth time (when the seed was created), β identifies the source query ID (which query generated this seed), and ρ records the successful reuse count. The anchor set $\mathbf{z}$ serves as the starting points for subsequent reuse, while the remaining metadata summarize freshness, origin, and past effectiveness, respectively. Among them, β and ρ are used directly during seed retrieval, whereas n_b is primarily used later for seed retention and eviction.

Table 2: Symbols used in BriskSeed design.

Symbol	Description
<i>Storage and Retrieval</i>	
q	Incoming query.
$\phi(q)$	Query signature for semantic routing.
$g(q), s(q), \mathcal{N}(q)$	Hot-key, semantic slot key, and probed semantic neighborhood.
S, m, P, H	Semantic slot count, per-slot seed upper limit, probing-width limit, and hot-layer capacity.
$r = \langle \mathbf{z}, n_b, \beta, \rho \rangle$	Record of a seed: historical top- k IDs, birth time, source query ID, and accepted-reuse count.
$u(r)$	Priority score of seed r for seed ranking.
<i>Validation</i>	
$d(\cdot, \cdot)$	Backend distance metric.
$\hat{\mathbf{z}}$	Recovered provisional top- k set.
$\text{qual}(q), \text{cons}(q)$	Candidate quality and overlap-consistency signals for acceptance.
<i>Writeback and Maintenance</i>	
$\text{keep}(r)$	Keep score of seed r for eviction.
$n, \text{age}_n(r)$	Processed-query counter and on-demand seed age $\text{age}_n(r) = n - n_b$.

Two-tier seed storage. Given the seed representation above, BriskSeed organizes reusable seeds in a two-tier storage, consisting of a *hot-exact* layer and a *semantic-shared* layer, to balance reuse quality and storage cost. The hot-exact layer is organized as a bounded key-value store with capacity H , where each query ID maps to a unique seed, enabling constant-time lookup for highly recurring queries. In contrast, the semantic-shared layer is organized as an array of S slots, each storing up to m seeds, to provide bounded long-tail coverage. To route queries to semantic slots, BriskSeed derives a lightweight query signature $\phi(q)$ from the query vector using a coarse projection (e.g., random projection or product quantization) that preserves approximate semantic information while remaining inexpensive to compute. The signature is used only for seed routing and does not participate in the ANN search.

A single-layer design is practically insufficient. Storing one seed per query in dedicated exact slots yields highly aligned retrieval but incurs prohibitive memory growth under dynamic workloads. Conversely, if all seeds are placed in a fully shared semantic array where queries are routed to shared slots via signatures, memory usage would be reduced but may return loosely matched seeds that require deeper multi-hop expansion, offsetting the benefit of reuse. The two-tier design balances these extremes: the hot-exact layer serves recurrent, high-value patterns, while the semantic-shared layer provides compact coverage for remaining massive yet low-utility historical outcomes.

3.2.2 Seed-guided search.

Seed retrieval. At query time, BriskSeed first attempts to retrieve reusable seeds from the two-tier seed storage.

Given a query q , it computes two indexing keys:

$$g(q) = \text{id}(q), \quad s(q) = \text{hash}(\phi(q)) \bmod S. \quad (7)$$

The key $g(q)$ is used for exact lookup in the hot layer, while the key $s(q)$ routes the query to a semantic slot in the shared layer. If a matching entry is indeed found in the hot layer, BriskSeed directly reuses the associated seed. Because the hot-layer seed is keyed by the same query ID, it is tightly aligned with the current query.

Otherwise, BriskSeed probes a small neighborhood of semantic slots, because seeds in the primary slot alone may be insufficiently aligned with the current query.

$$\mathcal{N}(q) = \{s(q), s(q) + 1, \dots, s(q) + P - 1\}, \quad P \ll S. \quad (8)$$

i.e., the primary slot $s(q)$ together with its $P - 1$ consecutive offset probes. Here, P is a manually controlled upper bound on semantic probing width, so probing inspects at most P slots with up to m seeds each, yielding a bounded retrieval cost of $O(Pm)$. All candidate seeds in the union of probed slots are then assigned a lightweight priority score:

$$u(r) = \lambda \text{sim}(\phi(q), \beta) + (1 - \lambda) \rho. \quad (9)$$

Here, $\text{sim}(\phi(q), \beta)$ measures the match between the current query signature and the seed’s source signature and ρ records how many times the seed has previously led to successful reuse. Based on these scores, BriskSeed selects the top-ranked seed as the warm-start anchor.

Candidate Collection. Given a selected seed, BriskSeed performs bounded candidate recovery on the graph index using the seed’s result nodes as starting points. For hot-exact seeds, the retrieved seed is tightly aligned with the query, so BriskSeed adopts one-hop neighbor expansion by default. In this process, BriskSeed retains the nodes recorded in the seed and expands one hop from them to collect their immediate neighbors, and the union of both forms the candidate set. This preserves the historical top- k results while incorporating a local correction to account for recent batch updates. As we show later in the evaluation, this low-overhead one-hop design is sufficient to recover strong candidates in most cases.

For semantic-shared seeds, one-hop expansion may not always recover sufficiently strong candidates because the retrieved seed is only approximately aligned with the query. BriskSeed therefore first performs one-hop expansion and examines the best candidate obtained. If the best one-hop candidate is still not close enough to the query judged by a distance gate, recovery continues to a two-hop; otherwise, it stops after one-hop. Recovery is capped at two hops, since deeper exploration leads to rapid frontier growth and empirically yields diminishing returns.

Given the recovered candidate set, BriskSeed ranks candidates under the backend distance metric and truncates them to form a provisional top- k set, denoted by $\hat{\mathbf{z}}$. When neither memory tier yields a reusable seed, the system falls back to the baseline graph-based ANN search to produce the final

top- k results. Thus, seed reuse is opportunistic rather than mandatory.

3.2.3 Candidate Validation. The bounded recovery stage produces only a provisional top- k result $\hat{\mathbf{z}}$, which BriskSeed does not accept blindly. Instead, it applies a lightweight query-level acceptance-and-fallback policy based on two signals: candidate quality and historical consistency.

For candidate quality, let c_1 and c_2 denote the best and second-best candidates in the recovered set under the backend distance metric $d(\cdot, \cdot)$. BriskSeed computes the normalized winner-quality signal

$$\text{qual}(q) = \frac{d(q, c_2) - d(q, c_1)}{d(q, c_1) + \epsilon}, \quad (10)$$

where $\epsilon > 0$ is a small constant for numerical stability. This signal favors provisional results in which the best recovered candidate is not only close to the query, but also clearly separated from its closest competitor. A small value of $\text{qual}(q)$ indicates that the current recovered set does not yet provide a sufficiently reliable winner.

For overlap consistency, BriskSeed compares the top- k results of the recovered candidate set, denoted by $\hat{\mathbf{z}}$, with the historical top- k set $\mathbf{z}$ stored in the seed, and computes

$$\text{cons}(q) = \frac{|\hat{\mathbf{z}} \cap \mathbf{z}|}{k}. \quad (11)$$

This signal measures the overlap between the provisional top- k recovered for the current query and what the seed previously contributed. If overlap is low, the local structure has likely drifted, so reuse is unlikely to help and should be rejected.

The recovered result is accepted only when both signals satisfy their acceptance thresholds; otherwise, BriskSeed falls back to the baseline graph-based ANN search. This conjunction avoids two failure modes: high $\text{qual}(q)$ but low $\text{cons}(q)$, and high $\text{cons}(q)$ but low $\text{qual}(q)$. Therefore, BriskSeed keeps only provisional results with both strong local candidate quality and sufficient overlap with the seed’s stored top- k results.

3.2.4 Writeback mechanism and seed update. After each query finishes, BriskSeed writes the final top- k results back to update seed statistics and memory contents. This writeback is performed for every query, regardless of whether the result comes from the hot-exact path, the semantic-shared path, or the baseline fallback.

If the query already has an entry in the hot-exact layer, BriskSeed directly updates the corresponding key-value record under the same query ID. Otherwise, the resulting seed is inserted into the semantic-shared layer according to its semantic slot. If the target semantic slot is full, eviction is governed by the keep score

$$\text{keep}(r) = \xi \rho - (1 - \xi) \text{age}_n(r), \quad \text{age}_n(r) = n - n_b. \quad (12)$$

where n is the cumulative number of processed queries so far. Seeds with low keep score are evicted from the corresponding semantic slot to maintain it upper threshold. Over time, this writeback process also adjusts seed placement across the two

tiers. Promotion and demotion are decided when a seed is used, based on its successful reuse count ρ and age $\text{age}_n(r)$. Semantic seeds with high reuse utility and recent birth are promoted to the hot-exact layer, while hot seeds with low reuse utility or old age are moved back to the semantic-shared layer. Overall, this policy keeps high-value seeds in memory while gradually removing stale or ineffective ones.

3.3 Cost Model and Decision Model

To make BriskSeed practical under fixed memory and latency budgets, we use a lightweight online decision model to set the hot/semantic split and probing parameters.

Budget split between hot and semantic layers. Let B be the total seed budget (in number of seed records). We split it as

$$H = \lfloor \alpha B \rfloor, \quad Sm = B - H, \quad (13)$$

where $\alpha \in (0, 1)$ is the hot-layer ratio, H is hot-layer capacity, and S, m are semantic slot count and per-slot cap. Thus, α directly controls the trade-off: larger α favors exact reuse for highly recurring queries, while smaller α favors broader long-tail coverage.

Expected query cost model. For one query, define: p_h as hot-hit rate, p_s as semantic-hit rate when hot misses, and p_a as acceptance rate after seed-guided recovery. Let C_h be hot-hit path cost, C_s semantic-reuse path cost (including probing/recovery/validation), and C_b baseline native-search cost. Then the expected cost is

$$\mathbb{E}[C] = p_h C_h + (1 - p_h) \left(p_s (p_a C_s + (1 - p_a) C_b) + (1 - p_s) C_b \right). \quad (14)$$

This form makes the decision objective explicit: maximize p_h and $p_s p_a$ while keeping C_s bounded by probing and recovery budgets.

Decision objective and constraints. We choose (α, P, m) to minimize $\mathbb{E}[C]$ under two constraints:

$$\text{Recall} \geq \tau \cdot \text{Recall}_{\text{baseline}}, \quad (15)$$

$$Pm \leq K_{\max}, \quad (16)$$

where τ is the recall ratio target and $K_{\max}$ is a fixed retrieval budget. The first constraint prevents over-aggressive reuse; the second keeps semantic-stage overhead bounded.

Online parameter setting in practice. We estimate $p_h, p_s, p_a, C_h, C_s, C_b$ from moving averages over recent rounds and update parameters every T rounds using a small candidate set. Specifically, we evaluate candidate α values (e.g., $\{0.2, 0.3, 0.4, 0.5\}$) and candidate (P, m) pairs satisfying $Pm \leq K_{\max}$, then pick the lowest-cost feasible configuration. This yields a stable and low-overhead controller: hot/semantic partition adapts to workload skew, while probing depth and per-slot fan-in remain bounded.

3.4 Time Complexity and Feasibility Analysis

We next analyze whether BriskSeed can reduce query-time cost in principle, using HNSW-style graph search as the reference baseline. Let $C_{\text{hns w}}$ denote the average distance-computation cost of one baseline query under a fixed recall target.

Per-query overhead of BriskSeed. For one query, BriskSeed adds four lightweight stages before deciding reuse or fallback. (i) *Seed lookup.* Hot-layer lookup is constant-time, while semantic probing inspects at most P slots and at most m seeds per slot, yielding $O(Pm)$ seed inspections. (ii) *Seed scoring and selection.* Ranking the inspected seeds is also bounded by $O(Pm)$. (iii) *Bounded candidate recovery.* Starting from top- k anchors, one-hop or guarded two-hop expansion is capped by design, so its work is bounded by a small constant factor times k and local graph degree, rather than a full best-first traversal. (iv) *Validation and writeback.* Computing $\text{qual}(q)$, $\text{cons}(q)$, and updating one seed record are linear in the recovered candidate size and thus bounded by the same capped frontier.

Hence, when reuse is accepted, BriskSeed query cost is dominated by bounded probing plus bounded local expansion, i.e., a capped local procedure instead of global graph exploration. Under fixed system budgets (P , m , and short-hop depth), this reuse path is $O(1)$ with respect to dataset size N .

Comparison to standard graph-based ANNS (HNSW). The original HNSW paper reports logarithmic complexity scaling with dataset size, while in practice per-query search cost is mainly controlled by the traversal budget needed for a target recall (typically reflected by $\text{texttt{Search}}$). In baseline HNSW, each query pays this traversal cost. In BriskSeed, an accepted reuse replaces the full traversal with a bounded local procedure. Let C_{reuse} denote the full bounded cost of one reuse attempt (lookup, scoring, capped recovery, validation, and writeback), and let p_{acc} denote the acceptance rate. Then BriskSeed’s expected query cost is upper-bounded by

$$\mathbb{E}[C_{\text{stream}}] \leq C_{\text{reuse}} + (1 - p_{\text{acc}}) C_{\text{hns w}}. \quad (17)$$

Therefore, a sufficient condition for latency improvement over native HNSW is

$$C_{\text{reuse}} < p_{\text{acc}} C_{\text{hns w}}. \quad (18)$$

This inequality directly explains when BriskSeed is faster: reuse itself must be cheaper than native traversal, and this advantage must occur often enough (high p_{acc}). Since C_{reuse} is capped by P , m , and short-hop recovery, the left-hand side remains bounded by design. Consequently, as N grows, BriskSeed preserves a bounded accepted-reuse cost while native HNSW search still grows approximately logarithmically; this creates an increasing asymptotic gap between one accepted reuse and one native search.

Table 3: Mapping from design aspects to implementation mechanisms.

Design Aspect	Implementation Focus
Workload-aligned reuse under drift	Bounded hint state, online utility-aware gating, warm-start seed adaptation, and fallback-on-drift safety.
Backend-decoupled control interface	Policy/Adapter/Runtime layering, backend-agnostic control signals, and runbook-configurable reproducible execution.

Feasibility under dynamic updates. Continuous updates mainly affect seed validity, not the complexity bound itself. BriskSeed keeps complexity stable through two mechanisms: bounded probing/recovery prevents unbounded per-query growth, and acceptance gating prevents low-quality reuse from replacing baseline search. As a result, dynamic drift lowers optimization gain primarily by reducing p_{acc} , while correctness is preserved by fallback and worst-case time remains comparable to baseline HNSW.

4 IMPLEMENTATION

To make the design-to-system path explicit, Table 3 summarizes how the two design aspects in Chapter 3 map to implementation focus in this chapter.

4.1 Algorithm for BriskSeed

Algorithm 1 gives the full online pipeline of BriskSeed, including seed retrieval, bounded recovery, acceptance/fallback, writeback, and online budget adjustment.

The logic is interpreted as follows. Line 1 defines all runtime inputs and control parameters. Lines 2–7 perform two-tier seed retrieval: first exact hot lookup, then bounded semantic probing and seed ranking. Lines 8–18 execute the seed-guided path, including bounded expansion, provisional top- k construction, and two-signal acceptance/fallback decision. Line 19 guarantees correctness when no reusable seed is available by directly invoking native graph search. Lines 20–21 perform online writeback and eviction, so seed memory stays bounded and utility-aware. Line 22 periodically updates hot/semantic budget and probing settings using recent telemetry, making the controller adaptive under drift.

4.2 Implementation for Design Aspect I: Workload-Aligned Reuse under Drift

To complement the pipeline in Section 3.2, we expose concrete decision thresholds and update rules in implementation. The goal is to keep reuse decisions stable and reproducible under drift, rather than tuning behavior implicitly.

Acceptance thresholds. BriskSeed accepts a provisional result only when both quality and consistency pass fixed gates. In our default setup, we use $\tau_{\text{qual}} = 0.08$ and $\tau_{\text{cons}} = 0.30$.

Algorithm 1 BriskSeed Online Pipeline

Require: Query q , graph index G , hot layer $\mathcal{H}$, semantic layer $\mathcal{S}$, and parameters $(P, m, \tau_{\text{qual}}, \tau_{\text{cons}})$

- 1: $r \leftarrow \emptyset$
- 2: **if** $g(q) \in \mathcal{H}$ **then**
- 3: $r \leftarrow \mathcal{H}[g(q)]$
- 4: **else**
- 5: Retrieve candidate seeds from $\mathcal{N}(q)$ with probing width P and per-slot cap m
- 6: Score each seed by $u(r) = \lambda \text{sim}(\phi(q), \beta) + (1 - \lambda) \rho$
- 7: $r \leftarrow$ top-ranked seed; if no seed is found, keep $r = \emptyset$
- 8: **end if**
- 9: **if** $r \neq \emptyset$ **then**
- 10: Recover candidates from seed anchors $\mathbf{z}$ by one-hop expansion
- 11: **if** r is semantic-shared and one-hop evidence is weak **then**
- 12: Continue to two-hop expansion
- 13: **end if**
- 14: Rank recovered candidates and form provisional set $\hat{\mathbf{z}}$
- 15: Compute $\text{qual}(q)$ and $\text{cons}(q)$
- 16: **if** $\text{qual}(q) \geq \tau_{\text{qual}} \wedge \text{cons}(q) \geq \tau_{\text{cons}}$ **then**
- 17: Accept $\hat{\mathbf{z}}$ as final output
- 18: Increase successful-reuse count ρ for seed r
- 19: **else**
- 20: Run native graph search on G and output top- k
- 21: **end if**
- 22: **else**
- 23: Run native graph search on G and output top- k
- 24: **end if**
- 25: Write back final top- k result as a new/updated seed record
- 26: If a target slot is full, evict by $\text{keep}(r) = \xi\rho - (1 - \xi) \text{age}_n(r)$
- 27: Every T rounds, update (H, S, m, P) from moving-average hit/acceptance/cost statistics

The first filters weak winners, and the second prevents low-overlap reuse from replacing native search.

Expansion trigger for semantic seeds. Semantic seeds use one-hop expansion by default and escalate to two-hop only when one-hop evidence is insufficient. Operationally, two-hop is triggered if one-hop fails either acceptance gate, i.e., $\text{qual}(q) < \tau_{\text{qual}}$ or $\text{cons}(q) < \tau_{\text{cons}}$. This keeps most queries on low-cost one-hop recovery while reserving deeper expansion for hard cases.

Bounded retrieval budget. We enforce the semantic probing budget through the hard cap $Pm \leq K_{\text{max}}$. In experiments, we set $K_{\text{max}} = 32$ and use $(P, m) = (4, 8)$ by default, which bounds retrieval overhead while preserving slot diversity.

Promotion and demotion rule. Tier movement is evaluated only when a seed is used. We promote a semantic seed if it has sufficiently high reuse count and low age, and demote a hot seed when reuse utility stays low over recent accesses.

To avoid oscillation, a moved seed is frozen for one update interval before becoming eligible for the next movement.

Periodic parameter refresh. Every T rounds (default $T = 10$), BriskSeed refreshes (H, S, m, P) using moving averages of hot-hit rate, semantic-hit rate, acceptance rate, and observed query cost. Only candidate settings satisfying the recall constraint and $Pm \leq K_{\text{max}}$ are considered, so adaptation remains safe by construction.

4.3 Implementation for Design Aspect II: Backend-Decoupled Control Interface

To operationalize the second design aspect, BriskSeed is integrated into CANDOR-Bench as a pluginized module with explicit separation between policy and backend mechanics. The implementation is organized into three cooperating layers:

- **Policy layer:** AQHC state machine, hint scoring, gating, and eviction.
- **Adapter layer:** translation between backend-agnostic hints and backend-native index operations.
- **Runtime layer:** runbook-driven scheduling, maintenance orchestration, and statistics logging.

This interface-first structure preserves existing experiment entry points while allowing backend-specific optimizations to evolve independently. Major controls (hint budget, gating thresholds, refresh aggressiveness, and maintenance cadence) are exposed through runbook and algorithm configuration files to ensure reproducibility. Consequently, the same policy implementation can be compared across event rates, datasets, and dynamic backends under consistent instrumentation.

5 EVALUATION

5.1 Setup and Overview

Goals. We evaluate BriskSeed along four questions: **EQ1 (Effectiveness)** asks whether BriskSeed improves end-to-end throughput under streaming updates while maintaining recall; **EQ2 (Robustness)** tests behavior under varying event rates, batch sizes, and distribution drift; **EQ3 (Ablation)** identifies which AQHC components contribute most to robustness; and **EQ4 (Overhead)** quantifies the amortized maintenance cost of hint control.

Workloads. We use streaming ANNS workloads with mixed query/update events over benchmark datasets (e.g., SIFT and additional stress variants). Runbooks control event rates, batch insertion scales, and maintenance cadence to emulate realistic online conditions. Our default reproducible setup uses algorithm `faiss_hnsw_streamseed`, dataset `sift`, runbook `runbooks/general_experiment/general_experiment.yaml`, and enables cache profiling.

Methods. We compare: (i) backend-native dynamic index without BriskSeed, (ii) BriskSeed-enabled backend, and (iii) selected ablation variants.

Table 4: Main results under default streaming runbook. Fill numbers after final migration run.

Method	Recall@ k	QPS	P95 Latency (ms)	Maint. Overhead (%)
Baseline	TBD	TBD	TBD	TBD
BriskSeed	TBD	TBD	TBD	TBD

Metrics. Primary metrics are Recall@ k , QPS, P95 latency, and amortized maintenance cost. When available, we additionally report cache-related statistics to explain systems-level behavior.

5.2 Main Results (EQ1)

Across default streaming settings, BriskSeed improves end-to-end search efficiency over the non-BriskSeed baseline while preserving competitive recall. Table 4 summarizes representative results.

5.3 Sensitivity and Drift Robustness (EQ2)

Under higher event rates, baseline performance degrades as reusable optimization context becomes stale. BriskSeed maintains more stable QPS and latency by adapting hint usage and refresh behavior. Figure-level analyses should report trends over event-rate and batch-size sweeps.

5.4 Ablation Study (EQ3)

We isolate AQHC modules by disabling one component at a time:

- without online usage gating,
- without hint-state updates,
- without confidence-aware eviction.

Results demonstrate that joint control is necessary; partial designs are less robust under drift.

5.5 Overhead Breakdown (EQ4)

We decompose runtime into search, hint lookup/gating, and maintenance updates. The additional policy cost remains bounded and is compensated by reduced ineffective traversals. This confirms that BriskSeed improves amortized end-to-end efficiency rather than only per-query micro-latency.

5.6 Discussion

Two practical observations emerge. First, bounded hint memory is sufficient when policy quality is high. Second, graceful fallback is essential for recall safety in rapidly changing regions. These findings guide deployment defaults for dynamic production workloads.

6 RELATED WORK

6.1 Most Directly Related: Dynamic and Graph-based ANNS

Graph-based methods such as HNSW and its variants are widely adopted for high-recall and low-latency vector search.

Most designs, however, are optimized for static or slowly changing datasets, where index quality and auxiliary search artifacts are built offline. Recent dynamic ANNS studies explore incremental index maintenance and update-aware operations. In contrast, BriskSeed focuses on policy-level control of query hint reuse and provides a pluginized path to integrate with existing dynamic backends.

6.2 Indirectly Related: System and Algorithm Acceleration

Prior system works improve ANNS throughput via hardware-conscious layouts, vector quantization, and SIMD-friendly distance computation. These approaches are complementary to BriskSeed: they optimize execution efficiency but do not directly address online staleness of reusable query guidance. Algorithmic methods reduce unnecessary distance evaluations through pruning, lower-bound estimation, or partial-dimension strategies. While effective in static contexts, they usually rely on assumptions that weaken under high-rate updates and distribution drift.

6.3 Other: Positioning of This Work

BriskSeed is orthogonal to both system-only and algorithm-only accelerations. Its contribution is a unified online control loop (AQHC) that jointly manages hint construction, usage, and update, bridging algorithmic reuse opportunities with systems-level deployability in streaming settings.

7 CONCLUSION

This paper presented BriskSeed, a pluginized optimization module for dynamic ANNS under streaming updates. By introducing Query Hint Reuse (QHR) with Adaptive Query Hint Control (AQHC), BriskSeed unifies hint construction, online usage control, and state updates within a single policy loop. The proposed design addresses the stale-hint challenge in dynamic workloads and improves amortized end-to-end efficiency while preserving recall safety through controlled fallback.

Implemented in CANDOR-Bench, BriskSeed enables runbook-driven and backend-comparable evaluation within a reproducible workflow. Empirical results and ablations indicate that unified hint control is a key ingredient for robust performance under high update rates and distribution drift.

Future work includes deeper backend co-design for dynamic HNSW internals, stronger drift-aware policy learning, and broader cross-dataset generalization studies.

ACKNOWLEDGMENTS

This work was supported by the [...] Research Fund of [...] (Number [...]). Additional funding was provided by [...] and [...]. We also thank [...] for contributing [...].

REFERENCES

- [1] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023* (Austin, TX, USA).

- (WWW '23). Association for Computing Machinery, New York, NY, USA, 3225–3235. <https://doi.org/10.1145/3543507.3583318>
- [2] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2, Article 137 (June 2023), 27 pages. <https://doi.org/10.1145/3589282>
 - [3] Yutong Gou, Jianyang Gao, Yuexuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 1, Article 80 (Feb. 2025), 26 pages. <https://doi.org/10.1145/3709730>
 - [4] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32.
 - [5] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
 - [6] Ziwen Song, Bin Wang, and Xiaochun Yang. 2025. Accelerating High-Dimensional ANN Search via Skipping Redundant Distance Computations. *Proc. ACM Manag. Data* 3, 6, Article 288 (Dec. 2025), 29 pages. <https://doi.org/10.1145/3769753>
 - [7] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proc. ACM Manag. Data* 3, 3, Article 123 (June 2025), 29 pages. <https://doi.org/10.1145/3725260>
 - [8] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen, Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 12 (2025), 5017–5030. <https://doi.org/10.14778/3750601.3750624>